SEMANA AI DATA ENGINEER · 2026

De Zero a Agente Autônomo

ShopAgent — Multi-Agent E-Commerce System

AIDE Brasil | 13 – 17 Abril 2026 | 20h – 23h

Instrutor: Luan Moreno

FORMAÇÃO AI DATA ENGINEER · aide-brasil.ai

A SEMANA

5 Noites, 1 Sistema Completo

SEG 13 · 20h

Ingerir

Dados fluindo via ShadowTraffic + Postgres + Qdrant. Claude Code assistindo. Pydantic & Structured Outputs. Agentic Commerce.

TER 14 · 20h

Contextualizar

Prompt Eng → Context Eng. RAG (Qdrant) + Ledger (Postgres). LlamaIndex pipeline. MCP conecta tudo. Claude Code pesquisa nos dados.

QUA 15 · 20h

Agente

Conceito de agentes e design patterns. AgentSpec: brainstorm → define → design → build. LangChain + Chainlit com interface conversacional.

SOBRE MIM

Luan Moreno M. Maciel

+17 anos transformando dados em decisões — de SQL Server on-premises a sistemas multi-agent autônomos em cloud.

2009	2013	2016	2017	2022	2025	2026
SQL Server	Microsoft MVP	PASS Summit	MS Ignite	Pythian	AI Agents	AIDE Brasil
Início da jornada	*Reconhecimento global*	*Copenhagen · SQL Nexus*	*Orlando · Tokyo*	*Cloud · Databricks*	*Claude Code · CrewAI*	*Formação · Semana*

Pythian

SOLUTIONS ARCHITECT

Modernização de pipelines legados para cloud-native. Databricks, AWS, Azure. Projetos enterprise globais.

InfraSight

PIPELINE ARCHITECT

Refatoração de DLT monolítico para serverless modular. 14.1x mais rápido. Paridade financeira exata.

Stealth Company

AGENTIC OS

Agentic Network Layer — sistema operacional para agentes autônomos de IA em escala.

Da Teoria à Produção

14.1x — PERFORMANCE
Mais rápido
InfraSight: de 6h05m para 25 minutos

Zero — PRECISÃO
Erros financeiros
Kurv: tolerância <0.01% em todas as fórmulas

632 — CONTEÚDO
Slides de produção
4 decks com SVG, animações e código

Claude Code
Production Engine

1.243 — QUALIDADE
Testes automatizados
Unit + integration. Coverage 80%+

110 — OTIMIZAÇÃO
API calls eliminadas
Data design removeu 110 chamadas/run

Meses → Horas — VELOCIDADE
File type onboarding
AgentSpec + TsysSpec: meses para horas

*Não foi protótipo – foi o **método de entrega** em produção.*

CLAUDE CODE | PRODUÇÃO | ENTERPRISE

PROJETO INFRASIGHT · LAKEFLOW SPARK

InfraSight

*Pipeline DLT monolítico com **5.449 linhas**. Uma falha paralisava o time inteiro. Refatoração para **serverless modular** com paridade financeira exata. Tudo com **Claude Code**.*

14.1x	32%	110	<0.01%
MAIS RÁPIDO	MENOS CÓDIGO	API CALLS CUT	TOLERÂNCIA

Built with **Claude Code** · Hybrid SQL+Python

Key Innovations

ARCHITECTURE DECISIONS

L — **5 Lambda Functions**
Serverless – Python 3.11 – SAM – EventBridge

HP — **Hybrid SQL+Python**
@dp.view() + CREATE OR REFRESH – solves JVM graph

ML — **InfraRating ML**
Linear regression UDF – scoring – paridade <0.01%

UC — **Unity Catalog**
UC Volumes – external locations – replaces mounts

CI — **DABs + GitLab CI/CD**
Asset Bundles – deploy automatizado – bundle validate

EventBridge → Lambda → Lakeflow

PROJETO KURV · PYTHIAN

Kurv in Action

Processadora de pagamentos com décadas de conhecimento em **T-SQL**. *Migração completa para* **AWS + Databricks** *sem perder um centavo de precisão. Tudo construído com* **Claude Code**.

The 5 Pillars

INTELLIGENCE SYMBIOSIS

TS — *TsysSpec* — 8
8-stage pipeline – 26+ artifacts – /tsys:build

AS — *AgentSpec* — 58
58 agents – 22 KB domains – 5-phase SDD

OS — *Agentic OS* — Opus
Claude Code – Opus 4.6 – 1M context – MCP

SDD — *Spec-Driven Development* — PRD
PRDs as contracts – clarity gates – spec before code

DK — *Data & Domain Knowledge* — 21.4K
specs/ – 12 docs – 4 shipped types – 21,423 records

From TSYS Files to Gold-Layer Analytics

01

TSYS FILES

Fixed-Width

- ADF
- MDI
- TDDF
- D256

4

file formats

58 record types

02

AWS LAMBDA

Parse & Transform

Detect file type
Parse records
COBOL overpunch
PII masking
Write Parquet

4 handlers

Python 3.12 · 1024 MB

O(1) streaming

21,422 records

Frozen dataclasses

03

S3 PARQUET

Staging

33 table types

Snappy compress

PyArrow schema

Partitioned by FileDate

Record distribution

04

QUALITY GATES

7 Gates

Lint · SAST
Unit + Coverage
SAM build
Parity check

1,243 tests

All passing

coverage

05

DATABRICKS LAKEHOUSE

Unity Catalog · Lakeflow DLT · Delta Lake

B	**Bronze — Raw Ingest** 39 tables · Auto Loader	**176,808**
S	**Silver — Quality + DQ** 15 tables · Quarantine · PII re-mask	**106,643**
G	**Gold — Analytics Ready** 5 views · Star Schema · OBT pattern	**15,820**

59 Delta Tables

< 35 seconds end-to-end

Mainframe fixed-width files become analytics-ready Delta tables — fully automated, zero manual intervention

SFTP → Lambda → S3 → Quality Gates → Bronze → Silver → Gold · Built with Claude Code · Pythian · Cleveland OH

Até 2030, agentes de IA vão intermediar mais comércio do que todos os marketplaces juntos. Nesta semana, vocês vão construir um desses agentes.

SHOPAGENT — SEMANA AI DATA ENGINEER 2026

BLOCO 1 · AI DATA ENGINEERING

O Novo Data Engineer

CONTEXTO DE MERCADO

O mercado *já decidiu*

Não é hype. São trilhões sendo redirecionados para agentes de IA agora. O Google lançou Gemini Enterprise for CX, a McKinsey prevê $3-5 trilhões em valor gerado, e 60% das buscas de produto já começam em interfaces de IA.

Antes vs. Agentic Commerce

O Funil Tradicional

O cliente pesquisa **manualmente**, compara opções, decide e compra. Dados de comportamento chegam **horas depois**. O Data Engineer analisa o passado.

- ✕ Funil de **7 etapas** sequenciais
- ✕ Dados em **batch** (D-1, D-7)
- ✕ Personalização por **histórico estático**
- ✕ Latência de insight: **horas a dias**

7 etapas

FUNIL TRADICIONAL

AI-Native Workflow

Agentes evoluem de assistentes passivos para **sistemas autônomos** que pesquisam, negociam e executam compras em tempo real.

- `real-time` Ciclo contínuo em **milissegundos**
- `vector` Personalização via **memória vetorial**
- `agent` DE constrói **sistemas que agem** sobre dados
- `stream` Dados em **streaming** em tempo real

1 conversa

AGENTIC COMMERCE

EVOLUÇÃO DO PAPEL

De Analista a Arquiteto de Agentes

ONTEM
SQL & Batch
Relatórios D-1, ETL agendado

→

HOJE
Streaming & Cloud
Pipelines em tempo real, data mesh

→

AGORA
IA Assistindo
Claude Code, Cursor, Codex como copilotos

→

AMANHÃ
IA Construindo
Você arquiteta, agentes auto-otimizam

Data Design para Agentes
Modelar dados para que agentes consumam com eficiência.

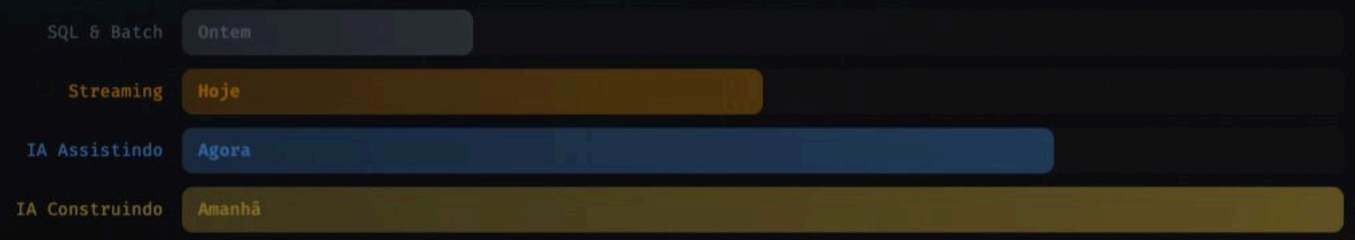
SKILL 01

Prompt & Context Engineering
Instruções que maximizam a qualidade do output do LLM.

SKILL 02

Orquestração Multi-Agent
Coordenar agentes especializados via CrewAI.

SKILL 03



BLOCO 2 · AI CODING AGENTS

AI Coding Agents

THE LANDSCAPE

From Language Models to Coding Agents

A IA está reescrevendo como software é construído. Os números contam a história.

THE EXPLOSION

GenAI & Coding Agents

*De chat completions a **agentes autônomos de código** que carregam conhecimento de domínio entre sessões.*

Claude Code — Agent Registry

```
# 10 specialized agents
> domain/ ......... shopagent-builder
> ai-ml/ .......... genai-architect
> ai-ml/ .......... ai-data-engineer
> ai-ml/ .......... prompt-specialist
> ai-ml/ .......... llm-specialist
> exploration/ .... codebase-explorer
> exploration/ .... kb-architect
> communication/ .. the-planner
```

6 KNOWLEDGE DOMAINS

langchain — Agent framework, tools	*crewai* — Multi-agent orchestration	*qdrant* — Vector DB, The Memory
supabase — Postgres, The Ledger	*deepeval* — LLM eval, testing	*pydantic* — Validation, structured output

> *Agentes não escrevem só código — eles carregam conhecimento de domínio entre sessões.*

DEFINIÇÃO

Autocomplete morreu. O agente chegou.

AUTOCOMPLETE — O PASSADO

Sugestões passivas

*Completa a linha que **você já começou a escrever**. Não entende o projeto. Não sabe o porquê. Apenas papagueia padrões estatísticos.*

- ✕ *Zero contexto — ignora arquitetura, deps, git*
- ✕ *Não testa, não valida, não corrige nada*
- ✕ *Você ainda faz **100% do trabalho pesado***

Tab, Tab, Tab

O MÁXIMO QUE FAZ POR VOCÊ

AI CODING AGENT — O PRESENTE

Execução autônoma

*Lê o projeto inteiro, entende a intenção, e **executa do início ao fim**. Escreve, testa, corrige, itera — sozinho. Você direciona, ele entrega.*

- OBSERVA — *Contexto completo: código, config, git, docs*
- EXECUTA — *Ações reais no filesystem, não sugestões*
- ITERA — *Falhou? Analisa, corrige, **tenta de novo***

Loop autônomo

ATÉ A TAREFA ESTAR COMPLETA

AS 4 FERRAMENTAS

Cada agente tem seu habitat

Cursor

IDE · VISUAL

Vive dentro do VS Code. Autocompletion, chat inline, edição assistida. Para quem quer IA sem sair do editor.

NA PRÁTICA

"Refatora essa função" – seleciona o trecho, edita inline com preview visual do resultado.

- ✓ *Autocomplete rápido*
- ✓ *Preview visual integrado*
- ✓ *Multi-modelo (Claude, GPT)*
- ✗ *Contexto limitado ao arquivo*
- ✗ *Não executa comandos*

Edição

MELHOR PARA EDIÇÃO PONTUAL

IDE NATIVO · VISUAL

Claude Code

TERMINAL · AGENTE

Vive no terminal. Lê o projeto inteiro, escreve arquivos, roda testes, corrige erros. O mais poderoso para projetos complexos.

NA PRÁTICA

"Cria o pipeline ETL com testes" – entrega pronto com commit, do zero ao deploy.

- ✓ *Contexto completo do projeto*
- ✓ *Executa + testa + itera*
- ✓ *MCP + SubAgents + KB*
- ✓ *Git, bash, filesystem real*
- ✓ *Loop autônomo até completar*

Construção

MELHOR PARA PROJETOS COMPLEXOS

HOJE À NOITE · PRINCIPAL

Codex

CLOUD · ASYNC

Roda na nuvem. Recebe tarefas, executa em background. Ideal para automação e CI/CD em escala massiva.

NA PRÁTICA

"Fix lint em 200 arquivos" – batch headless, 50 PRs por hora sem supervisão.

- ✓ *Execução assíncrona*
- ✓ *CI/CD nativo*
- ✓ *Sandbox isolado por task*
- ✗ *Sem interação em tempo real*
- ✗ *Só OpenAI (sem Claude)*

Automação

MELHOR PARA BATCH E CI/CD

CLOUD NATIVE · BATCH

Chat + Code

CHAT · INTERFACE

Interface conversacional. Upload de arquivos, análise de código, ideação. Ponto de entrada mais acessível para iniciantes.

NA PRÁTICA

"Explica esse erro de Python" – conversação livre, aprende rápido, cola código.

- ✓ *Baixa barreira de entrada*
- ✓ *Ótimo para aprender*
- ✓ *Upload de arquivos e imagens*
- ✗ *Não executa no filesystem*
- ✗ *Sem loop autônomo*

Exploração

MELHOR PARA APRENDER E IDEAR

EXPLORAÇÃO · ENSINO

MATRIZ DE DECISÃO

A ferramenta certa *para cada momento*

Cenário	Cursor	Claude Code	Codex	Chat	Veredicto
Escrever código novo *Feature completa, com testes e docs*	●	●●●	○	●	CC
Refatorar projeto inteiro *Renomear, mover, atualizar imports*	○	●●●	○	○	CC
Explorar dados ao vivo *SQL, DataFrames, análise exploratória*	●	●●●	○	●●	CC
Automação / CI batch *Pipelines headless, PRs automáticas*	○	●●	●●●	○	CDX
Aprender / Ideação *Entender conceitos, brainstorm, ensino*	●●	●	○	●●●	CHAT
Gerar docs, PRDs, specs *README, API docs, planejamento técnico*	●	●●●	○	●●	CC
Debug + troubleshooting *Rastrear bugs, ler logs, corrigir erros*	●●	●●●	○	●	CC

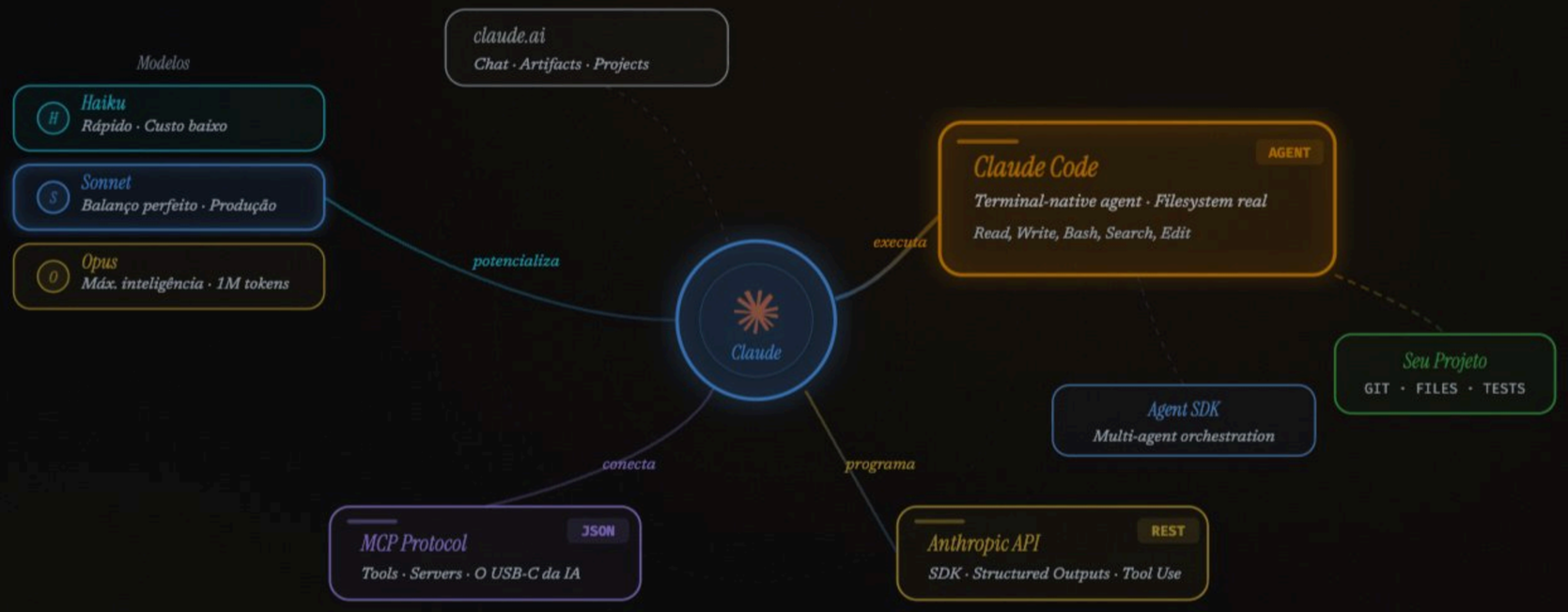
●●● Excelente ● Bom ○ Limitado CC = Claude Code CDX = Codex

BLOCO 3 · CLAUDE CODE

Claude Code *Deep Dive*

*A ferramenta que você vai usar **todas as noites** desta semana.*

Claude. Claude Code.



ANTHROPIC ECOSYSTEM · 2026

CLAUDE CODE

The Autonomous Coding Agent

*Um **agente terminal-native** que lê seu **codebase inteiro**, escreve código, roda testes e itera — **autonomamente**. Powered by **Opus 4.6** com janela de **1M tokens**.*

O que Claude Code faz

- *Explora e entende codebases inteiros*
- *Lê, escreve e refatora múltiplos arquivos de uma vez*
- *Roda bash, testes e builds autonomamente*
- *Memória persistente e knowledge bases entre sessões*
- *Spawna sub-agentes em paralelo via git worktrees*
- *Skills customizados, hooks e integrações MCP*
- *Cria PRs, revisa código e gerencia workflows git*
- *Disponível no Terminal, VS Code, JetBrains, Desktop e Web*

Claude Code — shopagent

```
$ claude
> Use o codebase-explorer para entender este repo

# Launching codebase-explorer agent ...
[explore gen/] Docker + ShadowTraffic
[explore src/] 4 days, Python modules
[explore .claude/kb/] 18 knowledge domains
[explore .claude/agents/] 10 sub-agents

Executive Summary:
ShopAgent — multi-agent e-commerce AI
Dual-store: The Ledger + The Memory
4 noites: Ingerir → Contextualizar → Agente → Multi-Agent

$ pytest --tb=short
42 passed in 3.2s ✓
```

LIVE DEMO FLOW · DIA 1

ShadowTraffic → Claude Code → Insights

STAGE 1

ShadowTraffic

GERADOR DE DADOS

Gera customers, products, orders e reviews em tempo real via Docker

→ *stream*

STAGE 2

Postgres — THE LEDGER

customers · products · orders

JSONL — REVIEWS

reviews.jsonl · sentimento · semântica

→ *lê*

STAGE 3

Claude Code

AGENTE AUTÔNOMO

Explora ambos os stores, cruza dados estruturados com semânticos, gera insights

SQL · RAG · ANÁLISE

→ *gera*

STAGE 4

SQL Insights

Ticket médio · Top produtos · Revenue por estado

RAG Insights

Sentimento · Reclamações · Padrões semânticos

POR QUE TERMINAL NATIVO

Sem context switch. Sem fricção.

O JEITO CHAT — FRICÇÃO

Copiar, colar, rezar

Você abre o chat, cola um arquivo, explica o contexto, recebe um snippet, ***cola no editor, testa, falha****, volta pro chat...*

- ✕ *Contexto perdido a* ***cada mensagem***
- ✕ *1 arquivo por vez —* ***nunca o projeto inteiro***
- ✕ *Não roda, não testa, não* ***corrige sozinho***
- ✕ *Você é o* ***middleware humano***

12+ switches

POR TAREFA SIMPLES

O JEITO TERMINAL — EXECUÇÃO

Um comando, resultado

Você descreve o que quer. Claude Code ***lê o projeto inteiro****, escreve, testa, corrige e entrega pronto.*

CONTEXTO	*Projeto completo na memória*
EXECUÇÃO	*Bash, git, testes — tudo no terminal*
ITERAÇÃO	*Roda, falha, corrige, repete sozinho*
MULTI-FILE	*Edita N arquivos em paralelo*

0 switches

UM FLUXO CONTÍNUO

0 *Context switches*
ZERO FRICÇÃO

100% *Contexto presente*
PROJETO INTEIRO NA MEMÓRIA

30s *Por 10K registros*
ANÁLISE COMPLETA

BLOCO 4 · CONFIGURAÇÃO AVANÇADA

MCPs, SubAgents & KBs

04

O protocolo universal que conecta agentes ao mundo real

Descoberta Automática

O cliente lista todas as ferramentas disponíveis no server via **tools/list**. Zero configuração manual de capabilities.

NA PRÁTICA:

```
claude > tools/list → execute_sql, semantic_search, create_pr ...
```

Protocolo Padronizado

JSON-RPC 2.0 sobre stdio ou SSE. **Qualquer linguagem, qualquer plataforma**, mesma interface universal.

NA PRÁTICA:

```
{ "jsonrpc":"2.0", "method":"tools/call", "params":{ ... } }
```

Segurança por Design

Cada tool call precisa de **aprovação explícita**. O agente pede, o protocolo media, o usuário controla.

NA PRÁTICA:

```
Allow tool execute_sql? [Y/n] → usuário aprova cada ação
```

Sem MCP, o agente é cego. Com MCP, ele executa.

SEM MCP

Claude Code isolado

Só consegue ler arquivos locais e rodar comandos no terminal. **Zero acesso a dados reais** do seu projeto.

- ✕ Não consulta banco de dados
- ✕ Não pesquisa vetores semânticos
- ✕ Não chama APIs externas
- ✕ Não gerencia infraestrutura

Chatbot

UM ASSISTENTE DE TEXTO

→

COM MCP

Claude Code conectado

Acessa **qualquer sistema externo** via protocolo padronizado. Executa queries, busca vetores, chama APIs.

- SUPABASE — SQL direto no Postgres
- QDRANT — Busca semântica em reviews
- GITHUB — PRs, issues, code search
- CONTEXT7 — Docs atualizadas de qualquer lib

Plataforma

UM SISTEMA QUE EXECUTA

KBs: memória de longo prazo para o agente

O COCKPIT COMPLETO

MCPs + KBs + SubAgents = Plataforma

HANDS-ON LIVE

Demo ao Vivo

DEMO AO VIVO — PASSO 1 DE 3

Explorar o *Projeto*

Claude Code vai ler o **CLAUDE.md**, *o* **init.sql** *e o* **shadowtraffic.json** *— e explicar cada pedaço da arquitetura.*

semana-ai-data-engineer — Claude Code

EXPLORER

- ▼ gen/
  - docker-compose.yml
  - shadowtraffic.json
  - init.sql
- ▼ docs/
- ▶ .claude/
- ▶ prompts/

● Claude Code — Opus 4

```
> Look at this project and explain what ShopAgent is. Read the CLAUDE.md to
understand the architecture. Give me a summary of:
  1. What we're building this week
  2. The data model (4 entities)
  3. The "Ledger + Memory" concept

* Reading CLAUDE.md…
ShopAgent is a multi-agent e-commerce AI system built across 4 live nights.
The Ledger (Postgres): exact data — revenue, JOINs
The Memory (Qdrant): meaning — sentiment, themes
✓ 4 entities: customers, products, orders, reviews
```

Ask Claude Code anything…

PROMPTS 01–03 | BLOCO 2 | EXPLORAÇÃO

BLOCO 5 · SHOPAGENT

ShopAgent — O Projeto

O caso de uso disruptivo que vocês vão construir esta semana.

“Até 2030, agentes de IA vão intermediar mais comércio do que todos os marketplaces juntos.”

MCKINSEY GLOBAL INSTITUTE · 2026

$3–5T
VALOR IA NO COMÉRCIO
McKinsey · até 2030
2030

Agent-First
NOVO PARADIGMA
Paradigma emergente
SHIFT

60%
PESQUISAS VIA IA
Google AI · pesquisas via IA
GOOGLE AI

O shopper do futuro não é humano. É um agente de IA que pesquisa, compara, negocia e executa — numa única conversa.

AGENTIC COMMERCE · SHOPAGENT · SEMANA AI DE

O agente precisa de duas pernas para andar

The Ledger

POSTGRES / SUPABASE

A planilha do contador – cada número é exato, determinístico, JOINável.

- *Dados estruturados: customers, products, orders*
- *Respostas exatas: somas, médias, contagens, JOINs*
- *A calculadora do agente – 100% preciso*

PERGUNTA:

"Quanto faturamos em SP?"

```
SELECT SUM(total) FROM orders
JOIN customers USING(customer_id)
WHERE state = 'SP';
```

✓ R$ 847.329,00

SQL | DETERMINÍSTICO | EXATO

+

The Memory

QDRANT / VETORES

O diário do cliente – sentimentos, temas, sinais que números não capturam.

- *Dados semânticos: reviews, reclamações, sentimento*
- *Busca por significado: padrões, contexto, nuance*
- *A intuição do agente – probabilístico, contextual*

PERGUNTA:

"Por que clientes de SP estão reclamando?"

```
semantic_search(
  "insatisfação entrega São Paulo"
)
```

✓ 72% reclamam de entrega

VETORES | SEMÂNTICO | RAG

ARCO DE CONSTRUÇÃO

4 Noites, 1 Sistema Completo

NOITE 01

Ingerir

ShadowTraffic gera dados sintéticos, Pydantic valida contratos, Claude Code orquestra.

SHADOWTRAFFIC PYDANTIC DOCKER

→

NOITE 02

Contextualizar

LlamaIndex ingere reviews, Qdrant armazena vetores, RAG responde com semântica.

LLAMAINDEX QDRANT RAG

→

NOITE 03

Agente

LangChain conecta tools, Chainlit entrega interface conversacional em tempo real.

LANGCHAIN CHAINLIT MCP

→

NOITE 04

Orquestrar

CrewAI coordena 3 agentes, DeepEval avalia qualidade, deploy na cloud.

CREWAI DEEPEVAL CLOUD

PROGRESSÃO DE AUTONOMIA

De copiloto a agente autônomo

01 *Eu faço*
IA OBSERVA

Autocomplete básico. IA sugere a próxima palavra. Você faz 100% do trabalho.

PASSADO

02 *Eu faço*
IA AJUDA

*Chat inline no IDE. IA responde perguntas e sugere blocos. Você ainda **dirige**.*

CURSOR

DIA 1 — VOCÊ ESTÁ AQUI

03 *IA faz*
EU REVISO

*O agente lê, escreve, testa e itera. Você **direciona e revisa**. Claude Code.*

CLAUDE CODE

04 *IA arquiteta*
EU VALIDO

*SubAgents com specs. IA define **a arquitetura**. Você valida decisões.*

DIA 3

05 *IA constrói*
IA EXECUTA

*Multi-agent crew. 3 agentes **decidem e executam** sem intervenção.*

DIA 4

BLOCO 6 · SHADOWTRAFFIC

ShadowTraffic — Dados Reais

GERAÇÃO DECLARATIVA DE DADOS

ShadowTraffic: *dados sem código*

O Gerador

Um container Docker que gera dados realistas declarativamente via JSON. Sem Python. Sem scripts. Só configuração.

- Zero dependências externas — 100% containerizado
- Volumes controláveis — de 100 a 1 milhão de registros
- Dados realistas via Faker integrado
- Relacionamentos via lookup — integridade relacional grátis

DOCKER NATIVO DECLARATIVO

```yaml
# docker-compose.yml
shadowtraffic:
  image: shadowtraffic/shadowtraffic
  volumes:
    - ./shadowtraffic.json:/home/config.json
    - reviews_data:/data/reviews
  depends_on:
    postgres:
      condition: service_healthy
```

4
ENTIDADES
customers, products, orders, reviews
Modelo relacional completo de e-commerce

2
DESTINOS
Postgres + JSONL
SQL estruturado + texto semântico

∞
VOLUME
Configurável por throttle
De 100 a 1 milhão de registros

Dois streams, dois destinos

Stream 1: Estruturado

POSTGRES · THE LEDGER

Tabelas SQL com colunas tipadas — cada campo é exato, relacional, JOINável.

- *customers (500), products (200), orders (contínuo)*
- *Formato: SQL tables, colunas tipadas, chaves estrangeiras*
- *ShadowTraffic grava direto no Postgres via conexão JDBC*

NA PRÁTICA:

```
ShadowTraffic → Postgres
  customers: 500 registros seed
  orders: geração contínua
```

✓ *Dados que o agente consulta com SQL*

SQL · POSTGRES · RELACIONAL

+

Stream 2: Não-estruturado

JSONL → QDRANT · THE MEMORY

Texto livre em português — sentimentos, temas, sinais que tabelas não capturam.

- *reviews (1.000): comentários em português, sentimento*
- *Formato: JSON lines, texto livre, embeddings depois*
- *ShadowTraffic grava em reviews.jsonl para ingestão no Day 2*

NA PRÁTICA:

```
ShadowTraffic → reviews.jsonl
  1000 reviews pré-gerados
  rating + comment + sentiment
```

✓ *Dados que o agente vai entender com vetores*

JSONL · SEMÂNTICO · RAG

CONFIGURAÇÃO JSON

Declarativo: descreva, não programe

shadowtraffic.json

```
{
  "generators": [{
    "topic": "customers",
    "row": {
      "customer_id": {"_gen":"uuid"},
      "name": {"_gen":"string",
        "expr":"#{Name.fullName}"},
      "segment": {"_gen":"oneOf",
        "choices":["premium","standard","basic"]}
    }
  },{
    "topic": "orders",
    "row": {
      "customer_id": {
        "_gen": "lookup",
        "topic": "customers",
        "path": ["row","customer_id"]},
      "status": {"_gen":"weightedOneOf",
        "choices":["delivered","shipped",
          "processing","cancelled"],
        "weights":[0.50, 0.20, 0.20, 0.10]}
    }
  }]
}
```

_gen: uuid
ID único e irrepetível para cada registro gerado
UUID

_gen: string + expr
Faker interpola expressões como #{Name.fullName}
FAKER

_gen: oneOf
Escolha aleatória uniforme entre valores definidos
RANDOM

_gen: lookup
Referência cruzada entre generators — integridade relacional
LOOKUP

_gen: weightedOneOf
Distribuição probabilística com pesos controlados
WEIGHTED

MODELO DE DADOS

4 entidades + 3 serviços Docker

500 REGISTROS

customers

uuid, name, email, city, state, segment

POSTGRES

200 REGISTROS

products

uuid, name, category, price, brand

POSTGRES

∞ STREAMING

orders

uuid, customer_id, product_id, qty, total, status, payment

POSTGRES

∞ STREAMING

reviews

uuid, order_id, rating, comment, sentiment

QDRANT

HANDS-ON LIVE

Dados ao Vivo

DADOS AO VIVO — PASSO 1 DE 4

Subir e *Explorar*

Containers rodando, dados fluindo. Claude Code vai consultar **Postgres** *ao vivo — counts, distributions, análise executiva.*

Terminal — semana-ai-data-engineer

```
$ docker compose up -d
[+] Running 3/3
✓ Container postgres  Started
✓ Container qdrant    Started
✓ Container shadowtraffic Started
```

```
> Connect to our local Postgres (localhost:5432, shopagent). Run queries:
  1. Count of rows in customers, products, orders
  2. Sample 5 customers…
  5. Distribution of customers by state
```

```
* Querying Postgres…
```

state	orders	faturamento
SP	312	R$ 48.290
RJ	187	R$ 29.150
MG	143	R$ 21.870

PROMPTS 04–07 | BLOCO 6 | POSTGRES + QDRANT

BLOCO 7 · PYDANTIC & OUTPUTS

Pydantic & Structured Outputs

PROBLEMA → SOLUÇÃO

JSON bruto é caos. Pydantic é ordem.

Sem Tipagem

*Dados brutos passam **sem validação** – erros silenciosos corrompem tudo downstream.*

```
# O que chega do ShadowTraffic
raw = {
  "order_id": "abc-123",
  "qty": "cinco",    # string!
  "total": -99.99,  # negativo!
  "payment": "boleto bancário"
}
# TypeError, KeyError, dados sujos
```

- × `qty="cinco"` causa TypeError em cálculos
- × `total=-99.99` viola regras de negócio
- × `payment` valor inválido corrompe análises silenciosamente

bugs silenciosos

CORROMPEM O PIPELINE

Com Pydantic

*Cada campo tem **tipo e constraint** – dados inválidos nunca passam.*

```
from pydantic import BaseModel, Field
from typing import Literal

class Order(BaseModel):
  order_id: str
  qty: int = Field(ge=1, le=100)
  total: float = Field(ge=0)
  payment: Literal[
    'pix','credit_card','boleto']

order = Order(**raw)  # Validado!
# ValidationError se inválido
```

- **RUNTIME** *Tipos garantidos em runtime – int, float, str*
- **CONSTRAINT** *Constraints de negócio declarativas – ge, le, Literal*
- **EXPLÍCITO** *Erros explícitos com ValidationError, não silenciosos*

zero surpresas

CONTRATO VALIDADO NA ENTRADA

MODELS & VALIDATORS

Cada entidade, *um contrato*

models/order.py

```python
from pydantic import BaseModel, Field, field_validator
from typing import Literal
from datetime import datetime
import uuid

class Order(BaseModel):
  order_id: str = Field(
    default_factory=lambda: str(uuid.uuid4()))
  customer_id: str
  product_id: str
  qty: int = Field(ge=1, le=100)
  total: float = Field(ge=0)
  status: Literal[
    'delivered','shipped',
    'processing','cancelled']
  payment: Literal[
    'pix','credit_card','boleto']
  created_at: datetime

  @field_validator('total')
  @classmethod
  def total_must_be_reasonable(cls, v):
    if v > 50000:
      raise ValueError('Total acima do limite')
    return round(v, 2)
```

Field Constraints

Limites numéricos declarativos: `ge`=maior-ou-igual, `le`=menor-ou-igual.

NA PRÁTICA:
`qty: int = Field(ge=1, le=100)`

Literal Types

Enum implícito — só aceita os valores listados, qualquer outro rejeita.

NA PRÁTICA:
`payment: Literal['pix','credit_card','boleto']`

@field_validator

Lógica customizada de validação por campo — Pydantic v2 usa `@field_validator` + `@classmethod`.

NA PRÁTICA:
`total > 50000 → ValueError`

ValidationError

Falha explícita com mensagem clara — nunca corrompe silenciosamente. Mostra ***campo, valor e regra violada****.*

NA PRÁTICA:
`1 validation error: total > 50000`

JSON tipado, não texto livre

HANDS-ON LIVE

Claude Gera Código

CÓDIGO AO VIVO — PASSO 1 DE 4

Construir e *Testar*

Claude Code gera **3 scripts Python** *ao vivo. E na última pergunta... descobre seu próprio limite.*

Claude Code — src/day1/models.py

```
> Read gen/init.sql and gen/shadowtraffic.json. Create Pydantic BaseModel classes for all 4 ShopAgent entities
in src/day1/models.py:
 Customer, Product, Order, Review
 Use Field validators matching DB constraints

* Reading init.sql + shadowtraffic.json…

class Customer(BaseModel):
  customer_id: int
  name: str
  segment: Literal["gold","silver","bronze"]
  # ... +3 more models
✓ Written to src/day1/models.py

> Which customers complain about late deliveries?

✗ I can read the reviews.jsonl file, but I can't search semantically…
This requires a vector database + embeddings.
```

PROMPTS 08–11 | BLOCO 7 | DIA 2 PREVIEW

PROVOCAÇÃO DIA 2

Imaginem 50.000 reviews de clientes. Como você encontra os padrões de insatisfação?

✖
Ler um por um
50.000 textos = impossível

✓
Busca por palavra-chave
perde contexto e sinônimos

✓✓✓
RAG + Qdrant
semântica real — Dia 2